

Connectivity Augmentation of Plane Graphs

Krishnan Dehaleesan
University of Bergen, Norway
krishnan.dehaleesan@uib.no

Asif Khan*
Chennai Mathematical Institute, India
asifkhan@cmi.ac.in

Pranabendu Misra
Chennai Mathematical Institute, India
pranabendu@cmi.ac.in

Abstract

We study the problem of connectivity augmentation of a planar graph, while preserving planarity. This problem is motivated by many real-world settings such as road-networks, power-networks etc. In these settings, it is crucial to preserve the original planar embedding after augmentation.

In 2009, Gutwenger and Mutzel gave a constructive algorithm showing that a connected planar graph with a fixed embedding (a plane graph) can be optimally augmented to a biconnected graph without crossings while preserving the embedding. We further this line of research, by giving an algorithm that computes a minimum set of edges that makes a connected plane graph 2-edge-connected in $O(|V|(1 + \alpha(|V|)))$ time and linear space, where α is the inverse Ackermann function.

We also study the 3-vertex-connectivity augmentation of biconnected outerplanar plane graphs. We present the first polynomial-time algorithm that augments such graphs to 3-connectivity with the minimum number of edges in $O(|V|(1 + \alpha(|V|)))$ time and linear space while preserving the embedding, i.e., the augmented graph has a planar embedding that extends the given embedding.

1 Introduction

The **connectivity augmentation** problem asks to increase the connectivity of a given graph to a specified level while adding as few edges as possible. This is a classic problem in graph algorithms and network design, motivated by applications in improving connectivity of real world networks, such as roads, electricity, telecommunication, and so on. While it is certainly possible to augment the connectivity by increasing the capacity of existing links, or adding parallel links, this is sub-optimal in both the costs, and in improving the network resiliency to link failures. For example, if a link in an electricity-network has failed due to some real-world incident (e.g a falling tree), it is highly likely that any parallel links would have also have failed. Hence, it is highly desirable to have connectivity via some alternative route.

This motivates the algorithmic study of connectivity augmentation problems, which play a fundamental role in the design of reliable communication and infrastructure networks. For general graphs, both biconnectivity augmentation (making a graph 2-vertex-connected) and bridge-connectivity augmentation (making a graph 2-edge-connected), with uniform costs for new-links, admit linear-time algorithms [5]. However, they become NP-hard when new links have different costs.

*Partially funded by a grant from Infosys foundation.

A variant of connectivity augmentation problems considers the case where the input graph is planar and we want the augmented graph to be planar as well. This is directly motivated by real-world settings such as augmentation of road networks, electricity networks, etc. that are always embedded on a plane, and hence a plane solution is highly preferred. This variant, called PLANAR BiCONNECTIVITY AUGMENTATION (PVCA), is known to be NP-hard [16], even with uniform new-link costs. The edge-connectivity version, PLANAR BRIDGECONNECTIVITY AUGMENTATION (PECA) is also NP-hard [18].

Let us observe that, in many motivating real-world problems, such as augmenting a road-network, not only is the input graph planar, but it has a fixed embedding of the nodes in the plane which cannot be changed. Therefore, it is highly desirable that upon augmenting the graph with new-links, the embedding is preserved.

This motivates us to study the following problem:

PLANAR BRIDGECONNECTIVITY AUGMENTATION FOR FIXED EMBEDDING (PECA-Fix)

Input: A connected planar graph G with a fixed embedding.

Task: Find a minimum set of edges $E' \subset V \times V$ such that $G + E'$ has a planar embedding that extends the planar embedding of G and is 2-edge connected.

The vertex-connectivity variant of PECA-Fix, called PVCA-Fix, can be defined similarly. This was studied by Gutwenger, Mutzel and Zey [9] who showed that solving biconnectivity augmentation for planar graphs with a fixed embedding (PVCA-Fix) is polynomial-time solvable in near-linear time. Since plane biconnectivity augmentation was poly-time solvable, it was natural to ask the question for connectivity greater than 2. Recent work by Akitaya et al. [1] showed that, even for plane graphs, increasing connectivity from c to k is NP-hard for all $2 \leq c < k \leq 5$. This motivated us to study the tri-connectivity augmentation of graphs from a subclass of planar graphs, namely outerplanar graphs, with a fixed embedding:

OUTERPLANAR TRICONNECTIVITY AUGMENTATION FOR FIXED EMBEDDING (OTA-Fix)

Input: An Outerplane biconnected graph G with a fixed embedding.

Task: Find a minimum set of edges $E' \subset V \times V$ such that $G + E'$ has a planar embedding that extends the planar embedding of G and is 3-vertex-connected.

Note that the requirement is only that the augmented graph admits a planar embedding extending the fixed embedding of G ; the augmented graph need not remain outerplanar. Indeed, except for a triangle, no 3-connected graph is outerplanar.

Let us note that the triconnectivity augmentation problem for outerplanar graphs (without a fixed embedding) is well-studied. The authors of [7] gave a polynomial time algorithm for the biconnectivity augmentation problem and the bridge-connectivity augmentation problem. For k even or $k = 3$, Nagamochi and Eades [17] provided optimal outerplanar edge-connectivity augmentation algorithms. Furthermore, Kant [15] provided an optimal solution in polynomial time for the Outerplanar triconnectivity augmentation problem.

The fixed embedding constraint rules out the known techniques for OTA-Fix. For instance, Kant's exact algorithm that augments an outerplanar graph to triconnectivity does not extend to the setting where the input graph has a fixed embedding. In particular, Kant's method may alter the embedding to a non-outerplanar one while preserving planarity, and then insert edges accordingly. To illustrate this limitation, consider the graph obtained from a cycle C_k by replacing each edge with a parallel edge and subdividing it. Kant's algorithm can make

this outerplanar graph 3-connected using $\frac{k}{2}$ edges by effectively moving vertices from the outer face into inner faces. Such modifications are not allowed in the OTA-Fix setting, where the embedding must be preserved. In fact, for the example above, no augmentation respecting the fixed embedding can achieve 3-connectivity using only $\frac{k}{2}$ edges.

This raises an interesting question: “how much worse can the solution be when the embedding is fixed?” For instance, in the case of 2-connectivity augmentation of plane graphs, there exist graphs where a fixed embedding requires k edges, whereas allowing changes to the embedding admits a solution of size $\frac{k}{2}$. Our results show that this gap is significantly smaller for outerplanar graphs: although the optimal augmentation without embedding constraints uses $\frac{k}{2}$ edges, in the fixed-embedding setting the optimum increases by at most one, i.e., it is bounded by $\frac{k}{2} + 1$.

1.1 Our Contributions

2-edge Connectivity Augmentation for Plane Graphs (PECA-Fix): We present the first polynomial-time algorithm to optimally augment a connected planar graph (with a fixed embedding) to eliminate all bridges. Our algorithm runs in $O(n \cdot (1 + \alpha(n)))$ time using $O(n)$ space, where $n = |V|$ and $\alpha(\cdot)$ is the inverse Ackermann function (arising from union-find operations). We build upon the framework of [9], meant for vertex-connectivity, to handle our edge-connectivity setting.

Triconnectivity Augmentation for Outerplane Graphs (OTA-Fix): Our main result is an $O(n \cdot (1 + \alpha(n)))$ time exact algorithm for augmenting a biconnected outerplanar graph (with fixed embedding) to achieve 3-vertex-connectivity.

In a setting where problems are NP-hard for connectivity greater than 2, we prove that Outerplane graphs form a sufficiently rich restriction admitting a fast polynomial time algorithm. In addition, We show that despite the embedding constraint, the optimal solution size increases by at most one in comparison with the analogous problem without a fixed embedding.

A central idea in our algorithm for OTA-fix is a face-splitting technique that could possibly be useful in other contexts as well. We identify a “large” face in the embedding that hinders connectivity, and we add internal edges to split this face into smaller ones, which simplifies the augmentation task. Moreover, for the outerplanar augmentation problem, we prove a noteworthy structural property of optimal solutions: at most two new edges need to be added inside the outerplanar graph in any optimum augmentation, no matter how large the graph is. This property significantly restricts the search space of possible solutions and underpins the correctness and optimality of our algorithm.

1.2 Related Work

Biconnectivity augmentation is the special case of k -connectivity augmentation for $k = 2$. Polynomial-time algorithms are known for $k = 3$ [12, 22] and $k = 4$ [13], while the problem remained widely open for $k \geq 5$. This was resolved when, for a fixed k , Jackson et al. [14] gave a polynomial-time algorithm, and Végő [20] showed that a $(k - 1)$ -connected graph can be augmented to k -connectivity in polynomial time. The analogous edge version includes bridge-connectivity augmentation ($k = 2$), and general k -edge-connectivity augmentation for which Watanabe [21] provided a polynomial-time algorithm.

For planar biconnectivity augmentation (PVCA), Kant and Bodlaender [16] gave a 2-approximation and claimed a $3/2$ -approximation; counterexamples were later shown by Fialko and Mutzel [6], who proposed a $5/3$ -approximation, which was also refuted by Gutwenger, Mutzel and Zey [8]. Thus, the best known ratio remains 2. Geometric variants of biconnectivity augmentation

have also been studied [3, 2, 19]. Other works on planar graphs include augmentation problems where the output is required to be regular (or cubic) [10, 11].

Organization of the paper. In Section 3, we give a polynomial-time algorithm for PECA-Fix in connected planar graphs with a fixed embedding. In Section 4, we study our main contribution to this paper: the OTA-Fix algorithm and its analysis. We first provide an algorithm that uses slightly more than the optimal number of augmentation edges in Section 4.3. Then, in Section 4.4 we use this to provide an exact polynomial time algorithm.

2 Preliminaries

For a graph G , $V(G)$ and $E(G)$ denote its vertex and edge sets. We use standard graph-theoretic terminology as in [4] for connectivity, cut-edges (also called bridges), cut-vertices and planar graphs. In particular, a graph is k -vertex-connected if it has more than three vertices and remains connected after deleting any set of at most $k - 1$ vertices. Similarly, a graph is k -edge-connected if it is connected and remains so even when any subset of edges of up to $k - 1$ size is deleted. A *plane graph* is a planar graph equipped with a fixed embedding in the plane.

A subset $S \subseteq V(G)$ is a *block* (or *biconnected component*) if $G[S]$ is biconnected and maximal with this property. A *bridge-component* is a maximal vertex set S such that $G[S]$ is 2-edge connected.

The *block-cut tree* (*bc-tree*) of a connected graph G , denoted $bc(G)$, is the graph whose nodes are the blocks (*b-nodes*) and cut vertices (*c-nodes*) of G , with a block node adjacent to a cut-vertex node iff the cut vertex lies in the block.

Similarly, the *bridgetree* $bt(G)$ has nodes corresponding to bridge-components (*b-nodes*) and cut-edges (*c-nodes*), where a bridge-component node is adjacent to a cut-edge node iff an endpoint of the cut-edge lies in the component. Every c-node in $bt(G)$ has degree 2.

The blocks corresponding to the leaf b-nodes in $bc(G)$ are called *pendant blocks*. Pendant bridge-components are defined analogously in $bt(G)$.

Definition 2.1 (Massive node & Balanced BC-tree). Given a connected graph G , let p be the number of leaves (pendants) in $bc(G)$. A c-node c^* in $bc(G)$, is called *massive* if $\deg_{bc(G)}(c^*) \geq \lceil \frac{p}{2} \rceil + 2$. The *bc-tree* of G is called *balanced* if it does not contain any massive c-node. Otherwise, $bc(G)$ is called *unbalanced*.

A bridge-component of G is called *pendant* if the node corresponding to it in the bridge-tree of G is a leaf.

Recall that G has a fixed planar embedding. Two bridge-components B_1, B_2 are *matchable* if they contain vertices on a common face, so that adding an edge between them preserves the embedding. The corresponding b-nodes in $bt(G)$ are also called matchable.

Observation 2.2. Let u, v be matchable pendant b-nodes in $bt(G)$ and let $G' = G + e$ where e joins vertices in components of u and v . Then $bt(G')$ is obtained from $bt(G)$ by contracting the u - v path in $bt(G)$ into a single b-node b' , preserving all adjacencies outside the path.

Observation 2.3 (Lower bound). If p is the number of pendants in $bt(G)$, at least $\lceil p/2 \rceil$ edges are required to make G 2-edge-connected.

For any two pendants p_1 and p_2 , adding the edge between them reduces the number of pendants remaining at least by 1. We shall now define a condition for bridgetrees analogous to the leaf connectivity condition in [9].

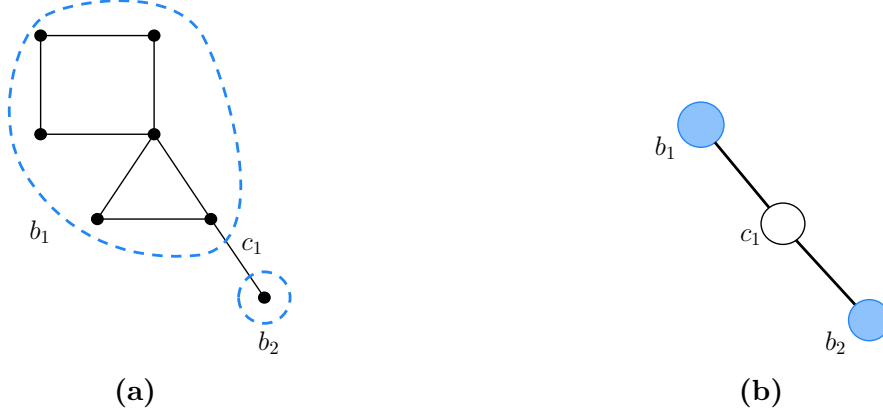


Figure 1: An example of representing a bt -tree. (a) Original graph with a fixed embedding; (b) The bridge tree of the graph.

Definition 2.4 (Leaf connection condition (l.c.c.)). Two pendants p_1, p_2 in $bt(G)$ satisfy the l.c.c. if the path between them contains either two b-nodes of degree at least three or one b-node of degree at least four.

Lemma 2.5. *If p_1, p_2 satisfy the l.c.c., adding an edge between them reduces the number of pendants by two.*

Bundles and labels were defined with the context of bc -tree in [6]. We shall do it for our requirement in the bt -tree now.

Bundles and labels. Let P be the set of pendants of $bt(G)$. A set $B \subseteq P$ is a *bundle* if (i) every pair in B is matchable, (ii) joining any pair produces a new pendant in $bt(G + e)$, and (iii) B is maximal. By Observation 2.2, two pendants lie in the same bundle iff the path between them in $bt(G)$ contains exactly one degree-3 b-node, called the *parent* of the bundle. A bundle together with its parent is a *label*; its size $L(\ell)$ is the number of pendants the bundle contains.

Intuition. Inside a bundle, any two pendants are mutually matchable and behave the same: connecting any pair does not eliminate the entire group, but produces a new pendant through their common parent. Thus, the bundle acts as a single unit attached to the rest of the bridgetree at its parent. Labels therefore partition the pendants into blocks that cannot be fully resolved internally; useful augmentation edges typically connect pendants from *different* labels, which ensures progress (via the l.c.c.) in reducing the number of pendants.

Lemma 2.6. *If two matchable pendants belong to different labels, then they satisfy the l.c.c.*

Proof. We prove that if p_1 and p_2 don't satisfy the l.c.c, then they must belong to the same label; this implies the statement of the lemma. If the l.c.c is not satisfied, then the path between p_1 and p_2 will have only one degree 3 b-vertex, v . In this case, adding an edge between p_1 and p_2 leaves the new coalesced block obtained from v with exactly one adjacent c-node and hence v is a new pendant, leaving us with only one less pendant. Hence, p_1 and p_2 belong to the same label. $\square$

3 Bridgeconnectivity augmentation for a fixed embedding

In this section, we give an algorithm that solves PECA-Fix. Recall that, in PECA-Fix, the input is a connected plane graph G and the goal is to find a subset of edges $E' \subseteq V(G) \times V(G)$, of minimum size such that $G' = G + E'$ is bridgeconnected and planar.

Input: A connected plane graph G
Output: A set of vertex pairs E' such that $G + E'$ is 2-edge connected
 $E' \leftarrow \emptyset$;
for each face F **do**
 $A \leftarrow \text{FACE_CONN}(F)$;
 $E' \leftarrow E' \cup A$;
return E' ;

Algorithm 1: PA_BRIDGECONN

Let us restate our result for the sake of convenience.

Theorem 3.1. *There exists an algorithm that for a given graph G solves PECA-Fix in $\mathcal{O}(|V(G)|(1 + \alpha(|V(G)|)))$ time and requires $\mathcal{O}(|V(G)|)$ space.*

3.1 Preliminaries and Structural Remarks

For a plane graph G which is a tree, we can provide a *cyclic ordering* of the leaves in this tree from its embedding. We can start from some leaf node and move along the outer face (the only face for a tree) in clockwise direction and add the leaves whenever they are visited. This provides a cyclic ordering, π , of the leaves in this tree. Two pendants are *adjacent* if they appear successively in the cyclic ordering. Two labels are said to be *neighbours* if a pendant of one is adjacent to a pendant of another. Each label has an adjacent label.

For a face F , consider the subgraph of G that consists of vertices and edges that lie on F , call it $\text{FACEGRAPH}(F)$ (for brevity, let $H := \text{FACEGRAPH}(F)$).

Since augmentation edges lie inside faces, the problem reduces to making each facegraph bridgeconnected. We therefore operate on the bridgetree of each facegraph. The embedding of $bt(H)$ is inherited from the embedding of H by preserving the cyclic order of incident cut edges at each block.

3.2 Algorithm for PECA-Fix

The algorithm works by computing the optimal edges to eliminate all the bridges in each face F . For each face F , Algorithm 2 computes H , and iteratively connects two ‘pendant’ components to reduce their number (hinging on the approach of [9] for biconnectivity).

Specifically, the following is done in H : we pick a maximum sized label and connect one of its pendant to an adjacent pendant from a different label. The algorithm keeps repeating this process over and over again until there is only one label, and then we handle one of two possible cases: the remaining label has size 3, or the remaining label has size 2. In each of the cases we find the optimal solution. Refer Figure 2 for an illustrative example of FACE_CONN .

Lemma 3.2. *PA_BRIDGECONN outputs a set of edges E' such that $G + E'$ is planar, extends the planar embedding of G , and is bridgeconnected.*

Proof. Each edge joins vertices on a common face and is inserted between adjacent pendant components in the cyclic order, so it can be drawn inside the face without crossings. The process continues until no cut-edge remains and the statement follows. □

What is left is to prove that within each face, our algorithm finds the optimal number of edges to add. We use this to conclude that the added edges in total comprise the optimal solution.

Input: A face F of a graph with a fixed embedding
Output: List of edges A that 2-edge-connect the face F
 $H \leftarrow \text{FACEGRAPH}(F)$;
 Compute $bt(H)$;
 $A \leftarrow \emptyset$;
while *number of labels* > 1 **do**
 Consider $bt(F)$ and take any label l_1 , and another neighbouring label l_2 ;
 Let p_1 from l_1 and p_2 from l_2 represent pendant blocks adjacent to each other in the
 cyclic ordering of pendants;
 Add an edge e in H connecting vertices of the blocks p_1 and p_2 ;
 Update $H \leftarrow H + e$;
 $A \leftarrow A \cup \{e\}$;
if *there is exactly one label l in H* **then**
 if $\text{size}(l) = 2$ **then**
 Let e be an edge connecting a vertex from each pendant block;
 $A \leftarrow A \cup \{e\}$;
 else
 if $\text{size}(l) = 3$ **then**
 Add 2 edges e_1 and e_2 between pendants p_1, p_2 and p_2, p_3 ;
 $A \leftarrow A \cup \{e_1, e_2\}$;
return A ;

Algorithm 2: FACE_CONN

Lemma 3.3. *For each face F such that $bt(\text{FACEGRAPH}(F))$ has p pendants, FACE_CONN uses $\lceil \frac{p}{2} \rceil$ (optimal) edges to 2-edge connect FACEGRAPH(F).*

Proof. Consider a face F , and let p denote the number of pendants.

If $p = 2$, Algorithm FACE_CONN connects the two pendants by adding one edge, regardless of whether their parent in the bt -tree is a b -node or a c -node. Thus, this case is resolved.

If $p = 3$, there must be a b -node of degree 3 in the bt -tree (since every c -node has degree 2). This is the only possible node of degree 3. By the contrapositive of Lemma 2.6, all three pendants have the same label. The algorithm therefore adds two edges to augment the graph.

Inductive hypothesis: For any bridgetree with $p' < p$ pendants, the face can be made bridgeconnected using $\lceil \frac{p'}{2} \rceil$ edges.

Now assume $p > 3$. Then there are at least two distinct labels (each b -label has size at most 3). The algorithm adds an edge between pendants of two different labels. By Lemma 2.6 and Lemma 2.5, this reduces the number of pendants by 2.

By the inductive hypothesis, $\lceil \frac{p'}{2} \rceil = \lceil \frac{p-2}{2} \rceil$ additional edges suffice to bridgeconnect the resulting graph $F + e$. Hence, the total number of edges used is

$$\left\lceil \frac{p-2}{2} \right\rceil + 1 = \left\lceil \frac{p}{2} \right\rceil,$$

which completes the proof. □

Combining the arguments in Lemmas 3.2 and 3.3 and arguing for the running time, we get the final theorem.

Theorem 3.1. *There exists an algorithm that for a given graph G solves PECA-Fix in $\mathcal{O}(|V(G)|(1 + \alpha(|V(G)|)))$ time and requires $\mathcal{O}(|V(G)|)$ space.*

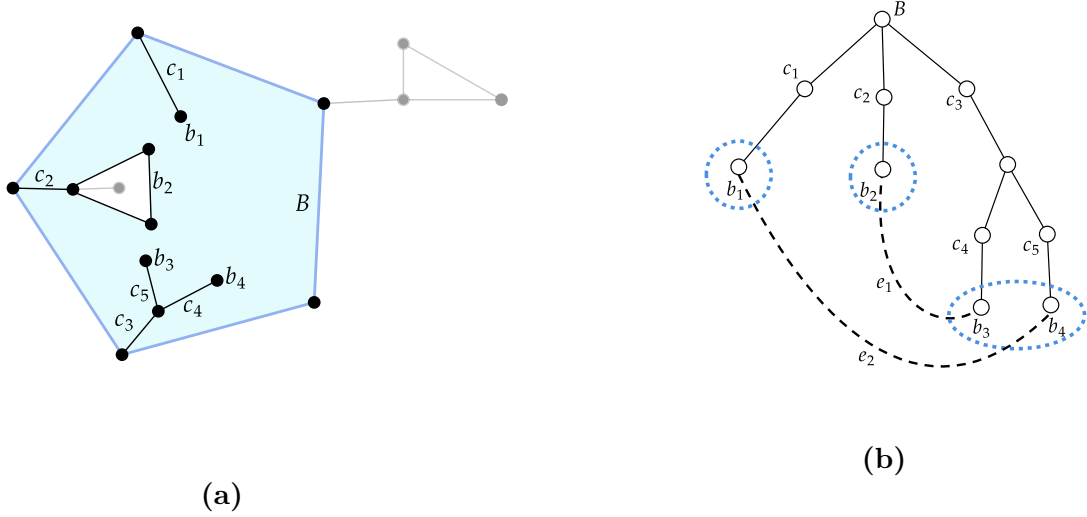


Figure 2: (a) A connected graph with a particular highlighted face containing cut edges (denoted by c_i 's) and bridge-components (denoted by b_i 's); (b) The bridge-tree decomposition of the FACEGRAPH, with dotted light-blue edges denoting the labels. The dotted black edges e_1 and e_2 show the augmentation edges between two neighbouring labels, in the first and second round of FACE_CONN respectively.

Proof. PA_BRIDECNN provides the correct solution (Lemma 3.3). We compute the initial *bt*-tree of each face F using a DFS-based decomposition in $\mathcal{O}(|V_F|)$ time, since planar graphs have a linear number of edges.

As edges are added during the augmentation, bridge-components merge. We maintain the *bt*-tree dynamically using a union-find data structure following [23]. Each bridge-component is represented as a set in the union-find structure. When an edge is inserted between two vertices belonging to different bridge-components, the corresponding sets are merged using a union operation. Since each edge insertion performs at most one union and a constant number of find operations, the total cost of maintaining component membership is $\mathcal{O}(|V_F|\alpha(|V_F|))$.

Importantly, the structure of the *bt*-tree changes only locally: inserting an internal edge can merge at most two bridge-components and affects only the path between them in the *bt*-tree.

Each bridge-component carries a label of size at most three. When two components merge, only their labels need to be updated, and since the label size is bounded by a constant, this update takes constant time. Furthermore, each edge insertion affects at most two labels, so label maintenance over all insertions is linear. Thus, all dynamic updates to the *bt*-tree and labels over the course of the algorithm require $\mathcal{O}(|V_F|(1 + \alpha(|V_F|)))$ time for each face F .

Since each face is processed independently and the sum of $|V_F|$ over all faces is $\mathcal{O}(|V(G)|)$ in a plane graph, the total running time is $\mathcal{O}(|V(G)|(1 + \alpha(|V(G)|)))$.

For space, we store the *bt*-tree, union-find structure, and labels, each of linear size. No step allocates more than linear auxiliary memory, so the total space usage is $\mathcal{O}(|V(G)|)$.

The procedure PA_BRIDECNN preserves the embedding and adds planar edges (Lemma 3.2). $\square$

4 Triconnectivity Augmentation of Outerplanar Graphs with a fixed Embedding

In this section, we turn to the main contribution of the paper and move to the domain of Vertex Connectivity Augmentation. Recall that in OTA-Fix, the input is a biconnected Outerplane graph G . We are required to find a minimum-size set of edges E' such that $G' = G + E'$ has a planar embedding that extends the original embedding.

Section and method overview Our strategy for solving OTA-Fix is split across various subsections. In Sections 4.1 and 4.2 we recall the concept of *treegraph* introduced by Kant [15] for outerplanar graphs and an algorithm that given a treegraph outputs a minimum set of edges that makes the treegraph biconnected such that the endpoint of every augmentation edge is a leaf vertex. Furthermore, we discuss the equivalences between the solution edges of the treegraph and solution edges of a biconnected outerplane graph. Finally, we discuss our novel contribution where we find an augmentation that is at most one edge away from optimal (Section 4.3), and then adjust it to an optimal solution in Section 4.4. The high-level idea is to handle separately the case of an ‘unbalanced’ face with odd number of degree-2 vertices by adding one or two interior edges inside a face of the graph.

4.1 Preliminaries

When we consider the input graph to be embedded on a plane, all the faces are *internal* except for one face which is *external* and unbounded. In similar vein, the edges that we add to augment the graph (edges in the solution) are of two kinds. Either they lie in an internal face, or they lie in the outer face. In case of the latter, it is called an *outer edge* otherwise, it is called an *inner edge*. Note that the distinction is made before any edges are added to the graph. For a given pair of non-adjacent vertices u, v in an internal face of the graph, the augmented edge uv can be an inner edge or an outer edge.

Before we proceed further, let us observe a lower bound on the size of a solution for OTA-Fix.

Observation 4.1. *For a given 2-connected outerplane graph G with k degree 2 vertices in the graph, we require at least $\lceil \frac{k}{2} \rceil$ edges to 3-vertex connect the graph.*

Let us now look at a *Tree* structure of the graph that was originally described in [15].

Definition 4.2. $T(G)$ or *Tree of the graph* for an outerplanar plane graph G is the interior dual of G with one pendant attached for each degree-2 vertex in G , embedded according to the cyclic order inherited from G .

Note that the interior dual for a biconnected outerplane graph is a tree. In the following, there is a change in notation; in the previous sections, we used p to denote the number of pendants. Let k denote the number of pendants for $T(G)$.

As an extension from previously used terminology, we define balanced graphs as follows based on balanced *bc*-trees

Definition 4.3. An outerplane 2-connected graph G is said to be balanced if $bc(T(G))$ is balanced.

Figure 3 provides an example of an outerplane graph and its corresponding treegraph.



Figure 3: (a) The graph G (black) and its dual graph (red), excluding the outer face. (b) The interior dual graph with a pendant attached for each degree-2 vertex, inheriting the canonical embedding from the embedding of G .

4.2 Plane Biconnectivity Augmentation for a Balanced Tree

In this subsection, we discuss an algorithm that outputs given a balanced tree, a minimum set of edges that makes the graph 2-vertex-connected ensuring that the output satisfies the additional properties stated in Lemma 4.7. Note that the reason we discuss the augmentation of a plane tree is revealed when it helps us with the augmentation of an outerplane biconnected graph as used in Sections 4.3 and 4.4.

The algorithm description is as follows:

Algorithm TREECONN:

Let G be a balanced plane tree. Compute its block-cut tree $bc(G)$ and the associated labels, and initialize $A := \emptyset$. While more than one label remains, select a label l_1 of maximum size and a neighboring label l_2 in the cyclic order induced by the embedding. Let $p_1 \in l_1$ and $p_2 \in l_2$ be pendant blocks that are consecutive in this cyclic order. Insert an edge e between simple vertices (non cut-vertices) of p_1 and p_2 , update $G \leftarrow G + e$, and add e to A .

When exactly one label l remains, if $|l| = 2$ then insert a single edge between simple vertices of the two pendant blocks. If $|l| = 3$ with pendant blocks p_1, p_2, p_3 , choose simple vertices $v_i \in p_i$ that are leaves of the original tree G , and add two edges $e_1 = v_1v_2$ and $e_2 = v_2v_3$ such that the endpoints of each edge do not share a common parent in G (i.e., neither $\{v_1, v_2\}$ nor $\{v_2, v_3\}$ are siblings). The algorithm outputs the set A of all inserted edges.

We recall a lemma that provides a sufficient condition when a c -node of high degree should be a parent of a label.

Lemma 4.4 ([9, Lemma 2]). *If a bc -tree $\mathcal{B}$ is not balanced, its massive c -node c^* must be the parent of a label, say ℓ_0 . Furthermore, ℓ_0 is the unique maximum label in $\mathcal{B}$.*

With this, we can proceed to prove the optimality of the algorithm with extra conditions.

Lemma 4.5. *Given a balanced plane tree G with p leaves, TREECONN computes an optimal augmentation of size $\lceil p/2 \rceil$ that makes G biconnected. Furthermore, every augmentation edge is incident only on leaves of G , and no augmentation edge joins two leaves with a common parent.*

Proof. We would like to proceed using induction. The bc -tree is balanced. For the base case of 2 pendants, $\deg(c^*)$ will be 2. Here, we augment the graph using one edge and we are done. If $p = 3$, then if it's from 1 label, we can add 2 edges inside the same label and complete. However, if we have 3 labels, we still only augment 2 edges from the algorithm. In both cases we've used $\lceil \frac{p}{2} \rceil$ edges.

Induction hypothesis: For $p' < p$ pendants we can augment using $\lceil \frac{p'}{2} \rceil$ edges

Since $p \geq 4$, there exists at least 2 labels (massive c -node otherwise). Hence, the algorithm adds a new edge between 2 different labels. Let l_1 and l_2 be the labels. Let l_1 be a label of maximum size.

Assume on the contrary that addition of the edge e creates a massive c -node c^* . Let $\deg(c^*)_{bef}$ and $\deg(c^*)_{aft}$ refer to the degree of c^* before and after the edge e is added respectively.

So, $\deg(c^*)_{aft} \geq \lceil \frac{p'}{2} \rceil + 2$. Assume that c^* was in the cycle created by the addition of the edge e . This would mean that $\deg(c^*)_{bef} = \deg(c^*)_{aft} + 1 \geq \lceil \frac{p'}{2} \rceil + 3 = \lceil \frac{p}{2} \rceil + 2$. However, this means that c^* was a massive node, contradicting the fact that there was no massive node.

Hence, assume that c^* does not lie in the cycle created by the edge added. We observe that $\deg(c^*)_{bef} = \lceil \frac{p}{2} \rceil + 1 = d$, say. Indeed, if $\deg(c^*)_{bef}$ were lesser, c^* can't become massive after the edge addition.

From Lemma 4.4, we see that c^* -label exists with maximum size (say j). Call this label l_0 . However we know that c^* is not in the cycle formed by the new edge. This means that The edge is added between two other labels l_1 and l_2 . However we required the edge to be added to maximum size label. Hence l_2 (WLOG) has size at least j . Remember that $\deg(c^*)_{bef} = d$ and the size j label accounts to j pendants. But the other $d - j - 1$ neighbours account for at least 2 pendants each since they are not part of the c^* -label. The remaining 1 neighbor accounts for the labels l_1 and l_2 with at least 1 and j pendants respectively. Now we count the number of pendants (p)

$$\begin{aligned} p &\geq 2j + 2(d - j - 1) + 1 \\ &= 2d - 1 \\ &= 2 \left(\left\lceil \frac{p}{2} \right\rceil + 1 \right) - 1 \\ &\geq p + 1 \end{aligned}$$

However, this is a contradiction. Hence, we can conclude that no massive c -node arises after the edge addition. Hence we can go through with the induction hypothesis to augment G' (the resultant graph) with $\lceil \frac{p'}{2} \rceil$ edges and our original graph with $\lceil \frac{p}{2} \rceil$ edges.

The algorithm makes sure that edges are not added between leaves with the same parent. Indeed, this is not possible when adding an edge between different labels. Assume that the algorithm has reached a point where there is exactly one label, the size of the label l is either $l = 3$ or $l = 2$. If $l = 3$, since G is not K_3 , for the 3 simple vertices v_1, v_2, v_3 in the respective bridgeblocks, we know that at most 2 of them share the same parent. Indeed, if all 3 vertices shared the same parent, then we would require another edge from the parent to be connected to the rest of the graph. However, this is not possible since the label has size 3. It is not possible that the entire graph is the 3 pendants and the parent since this is K_3 and we forbade it. Hence, with at most two vertices sharing the same parent, we can make sure to order them such that neither v_1, v_2 nor v_2, v_3 share parents.

We also posit that initially, the pendant blocks comprise the p degree 1 vertices as their only simple vertices (and their incident edge) in the tree G . Every time we add an edge between two different labels, the number of pendants go down by 2 and no new pendant block arises. Therefore, we are adding edges between leaves of the tree G . In case the graph only has 1 label, the construction still shows that we handpick the leaves from the pendants to add edges. Hence the set of augmentation edges only connects leaves of the graph G . $\square$

The arguments for the time and space finalises the analysis.

Lemma 4.6. *TREECONN can be implemented to run in $\mathcal{O}(1 + \alpha(|V(G)|)|V(G)|)$ time and linear space.*

Proof. We first compute the block-cut tree $bc(G)$ of the input tree G , which can be done in linear time. Using the leaf connectivity condition [9], we partition the pendants into labels in overall linear time.

When an augmentation edge is added, at most four labels are affected. Updating label membership and sizes therefore takes constant time per edge insertion.

We maintain the dynamic merging of components in $bc(G)$ using a union-find structure [23]. Each edge insertion triggers a constant number of FIND and UNION operations, yielding a total time of $\mathcal{O}(n \cdot \alpha(n))$ for all updates, where $\alpha(\cdot)$ is the inverse Ackermann function.

Since the algorithm processes each face independently and the total size of all facegraphs is $\mathcal{O}(n)$, the overall running time is $\mathcal{O}(n(1 + \alpha(n)))$.

For space, we store the bc -trees, union-find structure, and label data each requiring linear space. □

TREECONN is an augmentation procedure that provides a minimum set of edges satisfying some special properties. This is because we would eventually like the set of edges output by TREECONN to be used for triconnectivity augmentation of biconnected outerplane graphs. The specific properties are as stated in the lemma below:

Lemma 4.7. *For a balanced plane tree T with k leaves (that is not K_2 or K_3), we can compute in time $\mathcal{O}(1 + \alpha(|V|)|V|)$ and linear space an optimal augmentation A_T that biconnects T with the following properties:*

- $|A_T| = \left\lceil \frac{k}{2} \right\rceil$.
- Every edge in A_T is incident to a leaf of T .
- No edge in A_T joins two leaves of T that share the same parent.

Proof. This is a direct consequence of the Lemmas 4.5 and 4.6. □

Now we make the transition from the biconnectivity of a tree to the triconnectivity of an outerplane graph. For this reason, we need to show that the edges obtained while making $T(G)$ biconnected can be translated and used as the edges required to make G triconnected. For the case when G is balanced, the transition is direct.

Lemma 4.8. *For a given balanced outerplane graph G , we can compute A_{tri} in time $\mathcal{O}(1 + \alpha(|V|)|V|)$ and linear space such that $G + A_{tri}$ is triconnected and $|A_{tri}| = \left\lceil \frac{k}{2} \right\rceil$.*

Proof. Consider the tree of the graph G , $T(G)$. By using Lemma 4.7, we obtain A_T such that $|A_T| = \left\lceil \frac{k}{2} \right\rceil$ and $T(G) + A_T$ is biconnected. For every edge between 2 leaves in A_T , identify an edge between the corresponding degree 2 vertices in G . Let A_{tri} represent these set of edges. It is important to notice that there are no crossings and the planarity is preserved in the fixed embedding since A_{tri} doesn't have any crossings. We claim that $G + A_{tri}$ is triconnected.

Let $v_1, v_2, \dots, v_n$ represent the set of vertices in G in cyclic order. Let P_{v_i, v_j} represent the vertices $v_i, v_{i+1}, \dots, v_j$.¹

Consider two vertices v_i and v_j arbitrarily. We need to show that $G - \{v_i, v_j\}$ is still connected. We'll consider two cases.

¹when we say v_i we mean $v_{i \bmod n}$ since the ordering is cyclic

Case 1. v_i and v_j belong to different internal faces: In this case, on removing v_i and v_j , the path $P_{v_{i+1}, v_{j-1}}$ and $P_{v_{j+1}, v_{i-1}}$ still stays. But since v_i and v_j belong to different faces, there is a chord e between $P_{v_{i+1}, v_{j-1}}$ and $P_{v_{j+1}, v_{i-1}}$. Hence the remaining vertices are still all connected.

Case 2. v_i and v_j belong to an internal face: Consider an internal face with v_i and v_j . Removing v_i and v_j will give us two paths as before $P_{v_{i+1}, v_{j-1}}$ and $P_{v_{j+1}, v_{i-1}}$. Say v_i and v_j only have degree 2 vertices between them in the path. We can assume that v_i and v_j are not adjacent because G remains connected after their removal otherwise. Due to this assumption, let v be a degree 2 vertex in the arc $P_{v_{i+1}, v_{j-1}}$ in the face F . From Lemma 4.7, we know that in A_T , the vertex v is connected to a some vertex say u that doesn't share the same parent (F). Hence, v is connected to a vertex u from $P_{v_{j+1}, v_{i-1}}$. This means that $G - \{v_i, v_j\}$ is connected.

Otherwise, there are degree 3 vertices in $P_{v_{i+1}, v_{j-1}}$ and $P_{v_{j+1}, v_{i-1}}$ that are adjacent to other faces. Let vertices in each path can be grouped into faces (identify a face $F \in \mathcal{F}$ even if one vertex in the path P is incident on F). Let $\mathcal{F}_1$ and $\mathcal{F}_2$ represent the grouping of the faces from the paths $P_{v_{i+1}, v_{j-1}}$ and $P_{v_{j+1}, v_{i-1}}$ respectively. Notice that $\mathcal{F}_1$ and $\mathcal{F}_2$ are not empty from the very assumption of the case. Let $V(\mathcal{F}_i)$ represent the vertices corresponding to the faces in $\mathcal{F}_i$ for $i \in \{1, 2\}$. Since $T(G) - v_F$ is connected, there exists an outer edge e from some vertex $v_{F_1} \in \mathcal{F}_1$ to some $v_{F_2} \in \mathcal{F}_2$ that connects degree 2 vertices from $\mathcal{F}_1$ and $\mathcal{F}_2$.

WLOG, we have $v_a \in P_{v_{i+1}, v_{j-1}}$ and $v_b \in P_{v_{j+1}, v_{i-1}}$. $P_{v_{i+1}, v_{j-1}}$ and $P_{v_{j+1}, v_{i-1}}$ the paths are connected by e . Hence, the graph remains connected after the removal of v_i and v_j .

It is also important to notice that the planarity criteria is satisfied and no new crossings are formed. This is because the edges A_{tri} exactly correspond (in embedding) to the edges A_T and they have no crossings.

□

The implications of biconnecting the plane tree as shown has further consequences which shall be highlighted in Section 4.4. We shall now proceed to the algorithmic portion for solving OTA-FIX.

4.3 An Additive (+1)-Approximation Algorithm for Triconnectivity Augmentation of Outerplane Graphs

In this subsection, we present an algorithm for augmenting biconnected outerplane graphs that is almost optimal i.e, it might be off by an additional term of 1. The result is stated in the following lemma:

Lemma 4.9. *TRICONN triconnects an input biconnected outerplane graph $G = (V, E)$ with at most $\left\lceil \frac{k}{2} \right\rceil + 1$ edges (where k represents the number of degree-2 vertices in V) and runs in time $\mathcal{O}(|V|(1 + \alpha(|V|)))$*

We now briefly describe the structure of the algorithm used in this subsection.

Algorithmic overview. The algorithm TRICONN computes an augmentation that is at most one edge larger than optimal. For balanced instances, it invokes OUTERPLANARCONNECT, which is optimal. For unbalanced instances, it adds a carefully chosen edge inside a maximum-degree face so as to balance the graph, and then applies OuterPlanarConnect to the resulting instance.

Before we proceed further, we provide essential preliminaries and structural insights.

Preliminaries and Structural Results

Throughout the remainder of the section, we talk about internal faces in the graph and make a distinction between faces that are large and otherwise. Hence, it is essential and natural to introduce some new terminology.

Definition 4.10. For an Outerplane graph G with an internal face F , the *degree of a face F* or $d(F)$ refers to the degree of the vertex v_F representing the face in $T(G)$.

A face F is said to be *massive* if in $T(G)$ we have $d(v_F) - 1 > \left\lceil \frac{k}{2} \right\rceil$.

We observe that if a face has very high degree, it is not possible for the other faces in the graph to have a high degree.

Lemma 4.11. *Let G be an outerplane graph with k degree-2 vertices. If G contains a face F that is massive, then every other face F' satisfies $d(F') \leq \left\lceil \frac{k}{2} \right\rceil$.*

Proof. Let v_F be the node corresponding to a face F with $d(F) \geq \left\lceil \frac{k}{2} \right\rceil + 2$. Suppose for contradiction that there exists another face F' with node $v_{F'}$ such that $d(F') \geq \left\lceil \frac{k}{2} \right\rceil + 1$.

Consider the unique path between v_F and $v_{F'}$ in $T(G)$. Remove the edges of this path. The remaining graph consists of two subtrees rooted at v_F and $v_{F'}$, respectively. Each pendant belongs to exactly one of these subtrees or lies on the path.

The subtree rooted at v_F contains at least $\left\lceil \frac{k}{2} \right\rceil + 1$ pendants, since $d(F) \geq \left\lceil \frac{k}{2} \right\rceil + 2$ and at most one edge of the path is incident to v_F . Similarly, the subtree rooted at $v_{F'}$ contains at least $\left\lceil \frac{k}{2} \right\rceil$ pendants, since $d(F') \geq \left\lceil \frac{k}{2} \right\rceil + 1$.

Thus the total number of pendants in $T(G)$ is at least

$$\left(\left\lceil \frac{k}{2} \right\rceil + 1 \right) + \left\lceil \frac{k}{2} \right\rceil = k + 1,$$

contradicting the fact that G has exactly k degree-2 vertices, and hence $T(G)$ has exactly k pendants. $\square$

Now, we look at a structural lemma that guarantees a scenario where an edge added inside a face F balances the graph.

Lemma 4.12. *If G is an unbalanced graph with a massive face F such that adding e inside F leads to an old face F' in $G(\neq F)$ becoming the maximum degree face in $G + e$, then $G + e$ is a balanced graph.*

Proof. We know that the face F has a degree of $d(F) > \left\lceil \frac{k}{2} \right\rceil + 1$. We must have that $d(F') \leq \left\lceil \frac{k}{2} \right\rceil$ for all other faces $F' \neq F$ in G from arithmetic by counting the pendants from the fact that the total number of degree-2 vertices in G is k . Once the edge e is added, an old face F^* is the maximum-degree face. But we know that $d(F^*) \leq \left\lceil \frac{k}{2} \right\rceil$. That is, $d(F^*) - 1 \leq \left\lceil \frac{k-2}{2} \right\rceil$. $G + e$ has at least $k - 2$ degree-2 vertices (since the addition of an edge removes two degree-2 vertices at most). Hence, none of the faces are massive, which means that the resultant graph is balanced. $\square$

Algorithm for (+1)-approximation

The algorithm proceeds as follows: for unbalanced graphs, we cleverly pick an edge in the maximum-degree face that splits the face into two new faces of similar size and makes use of the subroutine for balanced graphs highlighted in Lemma 4.8.

Input: Outerplane 2-connected graph G

Output: List of edges E' such that $G(V, E \cup E')$ is planar and triconnected

Compute $T(G)$, $k \leftarrow$ no. of leaves, and $d \leftarrow \max_{v \in T(G)} d(v)$;

if $\lceil \frac{k}{2} \rceil \geq d - 1$ **then**

return OUTERPLANARCONNECT(G);

else

 Let v_F be the vertex corresponding to face F with maximum-degree d ;

 Let u be an arbitrary vertex in F (choose it to be a degree 2 vertex if it exists);

 Let v be another vertex in F such that $e = uv$ splits F into two faces F_1 and F_2 with

$|d(v_{F_1}) - d(v_{F_2})| \leq 1$;

return OUTERPLANARCONNECT($G + e$) $\cup \{e\}$;

Algorithm 3: TRICONN

Input: Outerplane 2-connected balanced graph G

Output: List of edges E' such that $G(V, E \cup E')$ is planar and triconnected

$A \leftarrow$ TREECONN($T(G)$);

Let A' represent the corresponding edge set in G from Lemma 4.8;

return A' ;

Algorithm 4: OUTERPLANARCONNECT

Although this solution does not claim to be the optimal one, it is indeed optimal when the input graph is balanced.

Lemma 4.13. *For a balanced outerplane 2-connected graph G , OUTERPLANARCONNECT computes an optimal triconnectivity augmentation in $\mathcal{O}(|V|(1 + \alpha(|V|)))$ time.*

Proof. If G is balanced, Lemma 4.8 implies that $\lceil \frac{k}{2} \rceil$ edges suffice and are necessary. The correctness and running time follow as well. $\square$

We can finally prove the correctness of the solution with the desired Time Complexity.

Proof of Lemma 4.9. If G is balanced, optimality and running time follow from Lemma 4.13. Assume G is unbalanced and let F be a face with $d(F) > \lceil k/2 \rceil + 1$. By Lemma 4.11, all other faces $F' \neq F$ satisfy $d(F') \leq \lceil k/2 \rceil$. Adding an internal edge e that splits F into faces F_1, F_2 with $|d(F_1) - d(F_2)| \leq 1$ ensures $d(F_i) - 1 \leq \lceil (k - a)/2 \rceil$ for $i \in \{1, 2\}$, where $a \in \{0, 1, 2\}$ counts the degree-2 vertices incident to e . Extrapolating from Lemma 4.12 and using the degree bounds on F_1 and F_2 , $G + e$ is balanced. Applying OUTERPLANARCONNECT yields a triconnected graph using at most $\lceil (k - a)/2 \rceil + 1 \leq \lceil k/2 \rceil + 1$ edges. The running time follows from the linear-time split and Lemma 4.13. $\square$

Although TREECONN might use one edge more than the optimal, in multiple instances, it does provide the optimal solution. In fact, there are various instances where $\lceil \frac{k}{2} \rceil + 1$ edges are required to make a graph triconnected.

4.4 An Optimal Algorithm for Triconnectivity Augmentation of Outerplane Graphs

In this subsection, we finally provide an optimal algorithm for the triconnectivity augmentation problem on general biconnected outerplane graphs (OTA-Fix) building on Sections 4.2 and 4.3. Let us restate it for convenience.

Theorem 4.14. *There exists an algorithm that solves OTA-Fix by running in time $\mathcal{O}(|V|(1 + \alpha(|V|)))$ time and linear space, augmenting $\text{OPT}(G) \in \{\lceil k/2 \rceil, \lceil k/2 \rceil + 1\}$ edges, where α denotes the inverse of the Ackermann function.*

Proof strategy. The additive-1 algorithm of Section 4.3 already shows that every instance can be augmented using at most $\lceil k/2 \rceil + 1$ edges. To obtain an exact algorithm, we first establish structural properties of optimal solutions. In particular, we show that unbalanced instances require internal augmentation edges when only $\lceil k/2 \rceil$ edges are used in an optimal solution. Moreover, we establish that all the internal edges can be added to the maximum-degree face (Lemma 4.18). These structural results allow us to distinguish instances whose optimum is $\lceil k/2 \rceil + 1$ from those admitting a solution of size $\lceil k/2 \rceil$. The resulting algorithm either identifies a balancing edge in the maximum-degree face or reduces the problem to a final balancing case.

The algorithm proceeds in stages. We first handle the case where the maximum-degree face contains at most four degree-2 vertices, for which an optimal augmentation can be obtained directly. We then consider several additional configurations that admit simple balancing steps or immediately imply that $\text{OPT}(G) = \lceil k/2 \rceil + 1$.

Finally, once these cases have been eliminated, the remaining instance satisfies a well-defined set of entry conditions. The algorithm first selects five well-spaced degree-2 vertices on the boundary of the maximum-degree face F . It then analyzes the facial segments determined by these vertices, and adds one or two internal edges to balance the graph. The resulting balanced instance is then completed optimally using OUTERPLANARCONNECT.

All edges inserted by the algorithm are drawn inside faces and are non-crossing, so the embedding of G is preserved throughout.

Before we proceed to the structural preliminaries required for this section, we record two consequences from our results in Section 4.2.

Supporting lemmas for Section 4.4

We observed that TREECONN provides a minimum edge set that helps us move from biconnectivity of a plane balanced tree to the triconnectivity of an outerplane balanced graph.

The following lemma shows that whenever an optimal augmentation using only outer edges exists for OTA-FIX, we may assume, without loss of generality, that all augmentation edges are incident only on degree-2 vertices.

Lemma 4.15. *If an outerplane biconnected graph G can be triconnected using $\lceil \frac{k}{2} \rceil$ outer edges A_{opt} (and no internal edges), then there exists A_{tri} such that A_{tri} only connects degree-2 vertices in G and $|A_{\text{tri}}| = \lceil \frac{k}{2} \rceil$ such that $G + A_{\text{tri}}$ is triconnected.*

Proof. Let G be considered to be an Outercircle C with interior chords. $G^* = G + A_{\text{opt}}$ is triconnected. If k is even, then all the edges in A_{opt} must connect degree 2 vertices in G . If k is odd, there exists at most one edge e connecting a degree 2 vertex u to a degree > 2 vertex, say, w . Since $\deg(w) > 3$, w should be incident on an internal chord, say, wx where x is another vertex. We can find a degree 2 vertex v in the Outercircle between the arc C_{wx} . Now, the vertices u, w, v, x are in cyclic ordering on C .

We claim that $A_{\text{tri}} = A_{\text{opt}} - \{uw\} + \{vw\}$ is an optimal solution as well. Assume on the contrary that in $G' = G + A_{\text{tri}}$ is not triconnected. So $G' - \{a, b\}$ is disconnected for some vertices a and b . Upon removal of a and b from G' , the circle C is split into 2 arcs P_1 and P_2 . If u and w are present in the same arc or if they are a part of the separating pair, we realize that since $G^* - \{a, b\}$ is not disconnected, there exists another edge e from P_1 to P_2 in G^* which should exist in G' as well. This ensures that P_1 and P_2 stay connected in G' thereby giving us a

contradiction that $\{a, b\}$ is not a separating pair. Hence, assume that u and w are in different arcs.

Hence, let a be in the arc C_{uw} ². The other vertex b can be in the arc C_{ux}, C_{xv} or C_{vw} . We'll show that for each possibility, there exists a contradiction.

If b is in the arc C_{ux} , then uv connects P_1 and P_2 . If b is in the arc C_{xv} , then either xw or uv connects P_1 and P_2 . If b is in the arc C_{vw} then xw connects P_1 and P_2 . Hence, we've arrived at a contradiction for all the cases exhausted and $G' - \{a, b\}$ remains connected. Since a, b were arbitrary, G' is triconnected. The running time and space complexity come from Lemma 4.7. $\square$

It is important to observe what happens when the graph is unbalanced. The following lemma and corollary discuss this.

Lemma 4.16. *If an outerplane biconnected graph G can be triconnected using $\lceil \frac{k}{2} \rceil$ outer edges (and no internal edges), then G is a balanced graph.*

Proof. Let A represent a set of optimal triconnecting edges for G , that is, $G + A$ is triconnected and $|A| = \lceil \frac{k}{2} \rceil$.

Due to Lemma 4.15 we can safely assume that for each edge in A , its endpoints are both of degree 2. Then, we have for each edge e in A , a counterpart on $T(G)$ (the edge between the two pendants in $T(G)$ that correspond to the degree 2 vertices that are endpoints of e in G). Then, we can see that the graph $T(G) + A$ is well defined. We claim that $T(G) + A$ must be 2-vertex connected.

Claim 1. *$T(G) + A$ is 2-vertex connected.*

Proof of claim. If not, let $v_F \in V(T(G) + A)$ be a cut vertex. Consider the bc-tree of $T(G) + A$ that is rooted at v_F . Let $a_1, a_2, \dots, a_m$ be the neighbours of v_F such that they appear in that cyclic order around v_F in the 'induced' embedding of $T(G)$.

Consider the bc-tree of $T(G) + A$ to be rooted at v_F . Then focusing on the neighbours of v_F , $a_1, a_2, \dots, a_m$ (that appear in that cyclic order in the embedding of $T(G)$), it must be the case that the partition of these vertices based on the connected component they belong to in $T(G) + A - v_F$ is a cyclic interval partitioning. That is, if $a_{i_1}, a_{i_2}, a_{i_3}, a_{i_4}$ appear in that cyclic order then, it can't be the case that a_{i_1} and a_{i_3} belong to one connected component and a_{i_2} and a_{i_4} belong to a different connected component of $T(G) + A - v_F$. This is because, all the edges in A are drawn externally in for the augmentation of G , and thus an interleaving will indicate violation of the planarity of $G + A$.

Now, we pick one connected component of $T(G) + A - v_F$. Let the vertices from $a_1, a_2, \dots, a_m$ that are picked in this connected component be $a_{i_1}, a_{i_2}, \dots, a_{i_\ell}$ (in the same cyclic order as in $a_1, \dots, a_m$). Our goal now is to find a separating pair in $T(G) + A$ to show a contradiction. For some $j \in [\ell]$, let $a_{i_j}, a_{i_{j+1}}$ be two vertices that do not consecutively appear around v_F . We identify a vertex s_x in G , for a_x (for $x \in \{a_{i_j}, a_{i_{j+1}}\}$) as follows. If a_x is a leaf vertex in $T(G)$ then s_x is the degree 2 vertex in G that corresponds to a_x . Otherwise, a_x is a face vertex that is adjacent to v_F . Exactly two vertices of this face lie on F as well. For a_{i_j} we pick the last vertex and for $a_{i_{j+1}}$ we pick the first vertex (first and last as per the cyclic order of vertices on the outer face in the embedding of G). We claim that these two vertices form a separating pair in $G + A$. And the reasoning is the following. There must exist an edge in A that connects the two distinct subtrees of v_F that contain a_{i_j} and $a_{i_{j+1}}$. This edge when drawn on the outerplane drawing blocks any pendant vertex that lies between s_x s (the two vertices we delete) to be connected

²Referring to the clockwise part of the cycle C restricted between a and b

beyond, since we do not have any edge to add internally, and external connection will violate planarity.

Hence $T(G) + A$ is 2-connected. The claim is proven. $\diamond$

Assume on the contrary that G is not balanced. So, $\max_v \deg(v) - 1 > \left\lceil \frac{k}{2} \right\rceil$ edges are required to augment $T(G)$ to biconnectivity. However, $T(G) + A$ is biconnected ($|A| = \left\lceil \frac{k}{2} \right\rceil$). This is a contradiction, and therefore, our assumption was wrong, and G is balanced. $\square$

From Corollary 4.17, unbalanced outerplane graphs requires at least 1 internal edge in any triconnectivity augmentation using only $\left\lceil \frac{k}{2} \right\rceil$ edges. In the upcoming section, we introduce how a minimum edge addition can make an unbalanced graph into a balanced one.

Corollary 4.17. *If G is an unbalanced outerplane graph, then $> \left\lceil \frac{k}{2} \right\rceil$ outer edges are required to make G 3-connected.*

Structural Preliminaries and Auxiliary Results

We introduce terminology and definitions that shall be used throughout the remainder of the section. For vertices u, v on the boundary of F , let P_{uv} denote the clockwise boundary path from u to v (and P_{vu} the complementary path). Inserting the edge uv splits F into faces F_1 and F_2 bounded by $P_{uv} \cup uv$ and $P_{vu} \cup uv$, respectively. Define $t(u, v) := d(F_1) - 1$ and $t(v, u) := d(F_2) - 1$.

Two degree-2 vertices u and v on the boundary of a face F are said to be *consecutive* along P_{uv} if all the interior vertices in P_{uv} have degree more than 2. We move to important structural insights.

Lemma 4.18. *There exists an optimal solution for OTA-Fix where all the internal edges are added inside the maximum-degree face.*

Proof. Let G be an input to OTA-Fix and let k be the number of degree-2 vertices.

If $bc(T(G))$ is balanced, then by Lemma 4.13, OUTERPLANARCONNECT yields an optimal solution using $\left\lceil \frac{k}{2} \right\rceil$ edges and no internal edges. Hence the statement holds trivially. Assume therefore that $bc(T(G))$ is unbalanced, and let F be a maximum-degree face.

First consider the case where the optimal solution has size $\left\lceil \frac{k}{2} \right\rceil + 1$. By Lemma 4.9, TRICONN computes an optimal augmentation of this size, inserting exactly one internal edge. This edge is placed inside F by construction, and thus the claim holds.

Let's assume now that the optimal solution uses $\left\lceil \frac{k}{2} \right\rceil$ edges, while $bc(T(G))$ is unbalanced: Let F represent the maximum degree face. We claim that there must be a degree 2 vertex in F . This is because, if this is not the case, then no internal edge can be added to F (any internal edge insertion would be suboptimal, because while it will not bring down the number of degree 2 vertices in G , but at the same time it will take up one edge from the budget of the optimal solution). So, we have established that F can't have any internal edges in the optimal solution. Now, consider all the internal edges (say, they are ℓ in number) that the optimal solution has to add to be already added. Let this intermediate graph be G' . G' must have either $k - 2\ell$ or $k - 2\ell + 1$ degree 2 vertices remaining. By assumption, since the optimal solution size is $\left\lceil \frac{k}{2} \right\rceil$, exactly $\left\lceil \frac{k}{2} \right\rceil - \ell$ are required to triconnect G' by insertion of only outer edges. Since G' is outerplanar, from Lemma 4.16, we have that G' must be a balanced graph. But, since F is the maximum degree face in G , and remains so in G' because all the other face degrees and number of degree 2 vertices has only gone down as compared to G , while degree of face F hasn't

changed. This means G' is still unbalanced, and we have a contradiction. So, from now on we can safely assume that F has at least one degree 2 vertex and moreover, any optimal solution (of size $\lceil \frac{k}{2} \rceil$) must have an edge added internally in F .

Now, if k is odd, then performing Algorithm 3 gives an optimal solution by adding an interior edge in F incident on a degree 2 vertex. Indeed, the remaining $k - 1$ (or $k - 2$) vertices are connected by $\lceil \frac{k-1}{2} \rceil$ (or $\lceil \frac{k-2}{2} \rceil$) edges so that in total $\lceil \frac{k}{2} \rceil$ edges are used. Hence, we are done in this case as well.

Consider the case when k is even. Let I be the set of edges in the optimal solution that are added inside F . I is not empty from the discussion above. Since k is even, each degree 2 vertex in F is incident on exactly a single edge from I . We can now assume that $bc(T(G + I))$ is not balanced. Because if it is not the case, then we can optimally connect the remaining degree 2 vertices in $G + I$ using OUTERPLANARCONNECT and we will obtain an optimal solution with all internal edges only being in F and would be done.

Let F' be the maximum degree face in $G + I$. It is unique since $bc(T(G + I))$ is unbalanced.

Case 1. F' is a new face:

By assumption, insertion of further internal edges does not split this maximum degree face F' . Hence, after all the internal edges have been inserted, the resulting graph still remains unbalanced. But, it can be triconnected optimally with only outer edges. This contradicts Corollary 4.17.

Case 2. F' is an old face:

$F' \neq F$ and is an internal face in G . Let $I = \{e_1, e_2, \dots, e_k\}$. For every $j \in [k]$ let $I_j = \{e_1, \dots, e_j\}$. Consider the following sequence of graphs, $G + I_1, G + I_2, \dots, G + I_k = G + I$. By assumption, F' is the maximum degree face in $G + I$. Let ℓ be the smallest index in $[k]$ such that F' is the maximum degree face in $G + I_\ell$. Then we claim that F' is the maximum degree face in all of $G + I_{\ell+1}, G + I_{\ell+2}, \dots, G + I_{k-1}$ as well. Indeed, the degree of F' doesn't change through these edge additions and no new face gets a high degree.

Let $G_1 = G + I_{\ell-1}$ and $G_2 = G + I_\ell$. In G_1 , the maximum degree face must be a face that lies inside the boundary of erstwhile F . In G_2 by assumption, F' becomes the maximum degree face. Let M be the face in $G + I_{\ell-1}$ to which the edge e_ℓ is added to get $G + I_\ell$. If G_1 is already balanced, we are done since we add the $\ell - 1$ edges to F and employ Lemma 4.13 to use $\lceil \frac{k}{2} \rceil$ edges in total. Otherwise, we can employ Lemma 4.12 to the graph G_1 upon adding the edge e since an old face gets the maximum degree after adding e . This gives us that G_2 is balanced.

In G_2 , there are $k - 2\ell$ degree 2 vertices remaining. Hence, use the ℓ edges from the face F and use OUTERPLANARCONNECT on $G + I_\ell$ to optimally augment the graph. We use $\ell + \lceil \frac{k}{2} \rceil - \ell = \lceil \frac{k}{2} \rceil$ edges exactly.

□

We can now show that for the case where the input graph G has an odd number of degree-2 vertices, the resulting augmentation is already optimal.

Lemma 4.19. *If k is odd, then TRICONN returns an optimal solution to OTA-Fix.*

Proof. Say the input graph G is balanced, then from Lemma 4.13, we know that the algorithm returns the optimal solution. If not, let F be the massive face. Say that face F has a degree 2 vertex. Then the algorithm adds an interior edge by connecting a degree 2 vertex and another vertex to split the face F . Once this edge is added, we know that the graph gets balanced.

Hence, in total, $1 + \left\lceil \frac{k-1}{2} \right\rceil$ or $1 + \left\lceil \frac{k-2}{2} \right\rceil$ are used to triconnect the graph depending on whether the edge e connects two degree 2 vertices or just one. Since k is odd, both the quantities are equal. Hence, an optimum of $\left\lceil \frac{k}{2} \right\rceil$ edges are used in total.

Now consider the case when F doesn't have a degree 2 vertex. We know from Lemma 4.18 that internal edges (if any) are added to F . If an internal edge was indeed added, we know that it doesn't connect any degree 2 vertex and k degree 2 vertices have to be connected using outer edges requiring at least $\left\lceil \frac{k}{2} \right\rceil + 1$ edges in total. If no internal edge was added, the graph is still unbalanced and from Corollary 4.17, at least $\left\lceil \frac{k}{2} \right\rceil + 1$ edges are required by the optimal solution. Hence, in totality, TREECONN provides the optimal solution. $\square$

Case when the optimum is $\left\lceil \frac{k}{2} \right\rceil + 1$

Going forward, we can assume that the input graph G has even number of degree-2 vertices, since otherwise Lemma 4.19 is applicable. Below, we discuss a structural case when at least $\left\lceil \frac{k}{2} \right\rceil + 1$ edges are required in an optimal solution.

Proposition 4.20. *Let G be an instance of OTA-FIX and let F be a face containing two consecutive degree-2 vertices v_1 and v_2 (i.e., no other degree-2 vertex lies on the boundary of F between them). If $t(v_1, v_2) \geq \left\lceil \frac{k}{2} \right\rceil$ and k is even, then at least $\left\lceil \frac{k}{2} \right\rceil + 1$ edges are required to triconnect G .*

Proof. Assume that the optimal solution has size $\left\lceil \frac{k}{2} \right\rceil$. From Corollary 4.17, since G is unbalanced, the solution must include internal edges.

Claim 2. *The graph G remains unbalanced after adding the internal edges.*

Proof of claim. Since k is even and the optimal solution uses $\left\lceil \frac{k}{2} \right\rceil$ edges, these edges can only connect the k degree-2 vertices. After inserting the internal edges, all vertices between v_1 and v_2 along the boundary of F lie on a common face F' . By definition of $t(v_1, v_2)$, we have

$$d(F') \geq t(v_1, v_2) + 1 \geq \left\lceil \frac{k}{2} \right\rceil + 1.$$

After the internal edge insertions, at most $k - 2$ degree-2 vertices remain. Hence,

$$d(F') \geq \left\lceil \frac{k}{2} \right\rceil + 1 \geq \left\lceil \frac{k'}{2} \right\rceil + 2,$$

implying that F' is still a massive face and the graph remains unbalanced. $\diamond$

By Lemma 4.18, we may assume that all internal edges of the optimal solution lie in F . Suppose l such edges are added. Then $k - 2l$ degree-2 vertices remain. Since the graph is still unbalanced, Corollary 4.17 implies that at least $\left\lceil \frac{k}{2} \right\rceil - l + 1$ outer edges are required to achieve triconnectivity. Therefore, the total number of edges used is at least

$$l + \left(\left\lceil \frac{k}{2} \right\rceil - l + 1 \right) = \left\lceil \frac{k}{2} \right\rceil + 1,$$

contradicting the assumption that the optimal solution has size $\left\lceil \frac{k}{2} \right\rceil$. $\square$

Input: Outerplane 2-connected graph G
Output: List of edges E' such that $G(V, E \cup E')$ is planar and triconnected
Compute $T(G)$, $k \leftarrow$ no. of leaves, and $d \leftarrow \max_{v \in T(G)} \deg(v)$;
if $T(G)$ is balanced or k is odd **then**
 return $\text{TRICONN}(G)$; // Lemma 4.13 and Lemma 4.19
Let v_F be the vertex corresponding to face F with maximum-degree d in G ;
if F has ≤ 4 degree 2 vertices on it **then**
 Brute-force internal (vertex-disjoint) edge addition in F with a budget of at most 2
 non-crossing edges between non-adjacent degree 2 vertices;
 if there exists a set of edges M such that $G + M$ is balanced **then**
 return $M \cup \text{OUTERPLANARCONNECT}(G + M)$;
 return $\text{TRICONN}(G)$; // Lemma 4.21
Let $v_1, \dots, v_r$ (with $v_{i \bmod r}$) be the degree 2 vertices in F in consecutive cyclic order;
for $i \in [r]$ **do**
 if $t(v_i, v_{i+1}) \geq \lceil \frac{k}{2} \rceil$ **then**
 return $\text{TRICONN}(G)$; // Proposition 4.20
 else if $t(v_i, v_{i+1}) = \lceil \frac{k}{2} \rceil - 1$ **then**
 return $\{v_i v_{i+1}\} \cup \text{TRICONN}(G + v_i v_{i+1})$; // Observation 4.22
Let u be a degree 2 vertex of F ;
if there exists a degree 2 vertex v in F such that $|t(u, v) - t(v, u)| \leq 1$ **then**
 return $\{uv\} \cup \text{OUTERPLANARCONNECT}(G + uv)$;
else
 return $\text{FIVEVERTICES}(G, u)$;

Algorithm 5: TRICONNEXACT

Easy Optimal cases

In some configurations, it is possible to save case analysis. We shall now focus on the case where it is easy to compute the optimal solution.

Lemma 4.21 (Small-face case). *Let F be a maximum-degree face containing at most four degree-2 vertices. Then TRICONNEXACT returns an optimal augmentation for OTA-Fix.*

Proof. Algorithm 5 enumerates all possibilities of inserting one or two internal edges between degree-2 vertices in F .

Suppose the optimal solution has size $\lceil \frac{k}{2} \rceil$. Since k is even, any optimal solution connects only degree-2 vertices. By Lemma 4.18, there exists an optimal solution whose internal edges all lie inside F . Hence, the brute-force step exhausts all possible internal-edge configurations of an optimal $\lceil \frac{k}{2} \rceil$ -edge solution, and the algorithm finds such a solution.

Otherwise, every set of internal edges leaves the graph unbalanced. By Corollary 4.17, any triconnectivity augmentation then requires at least $\lceil \frac{k}{2} \rceil + 1$ edges. In this case, TRICONN produces an optimal solution of size $\lceil \frac{k}{2} \rceil + 1$ (Lemma 4.9).

Thus, in both cases the algorithm outputs an optimal solution. $\square$

The remaining easy case where $t(v_i, v_{i+1}) = \lceil \frac{k}{2} \rceil - 1$ for two degree-2 vertices v_i and v_{i+1} occurs as a continuation of analysis from Proposition 4.20. Here, one internal edge suffices to balance the graph.

Input: Outerplane 2-connected graph G , a degree-2 vertex u in the maximum degree face F

Output: List of edges E' such that $G(V, E \cup E')$ is planar and triconnected

Compute $T(G)$, $k \leftarrow$ no. of leaves, and $d \leftarrow \max_{v \in T(G)} \deg(v)$;

Let v be a degree ≥ 3 vertex in F such that $|t(u, v) - t(v, u)| \leq 1$;

Let w, x be the degree 2 vertices in F encountered moving anticlockwise and clockwise from v along the boundary of face F ;

Let y, z be any two other degree 2 vertices on F ;

Let u_1, u_2, u_3, u_4, u_5 be the vertices u, w, x, y, z in consecutive clockwise cyclic order along the face such that $t(u_1, u_2) \geq \max_i t(u_i, u_{i+1})$;

if *there exists a face F' in G such that $d(F') = \lceil \frac{k}{2} \rceil$* **then**

return $\{u_2u_4\} \cup \text{OUTERPLANARCONNECT}(G + \{u_2u_4\})$; // Observation 4.24

$M \leftarrow \emptyset$;

Let the input in the given cyclic order be a_1, a_2, a_3, a_4, a_5 where $t(u_i, u_{i+1}) = a_{i+1}$ and $a_1 = \max_i a_i$;

if $a_1 = 0$ **then**

$M \leftarrow \{u_2u_4\}$;

else if $a_4 + a_3 \leq a_1 + a_3 + a_5$ *and* $a_2 + a_5 \leq a_1 + a_3 + a_4$ **then**

$M \leftarrow \{u_1u_5, u_2u_4\}$;

else if $a_4 + a_3 = a_1 + a_2 + a_5 + 1$ **then**

$M \leftarrow \{u_2u_4\}$;

else if $a_2 + a_5 = a_1 + a_3 + a_4 + 1$ **then**

$M \leftarrow \{u_5u_4, u_1u_3\}$;

else if $a_4 + a_3 \geq a_1 + a_2 + a_5 + 2$ **then**

$M \leftarrow \{u_1u_5, u_2u_3\}$;

else if $a_2 + a_5 \geq a_1 + a_3 + a_4 + 2$ **then**

$M \leftarrow \{u_1u_2, u_5u_4\}$;

return $M \cup \text{OUTERPLANARCONNECT}(G + M)$; // Proposition 4.27

Algorithm 6: FIVEVERTICES

Observation 4.22. *If $t(v_i, v_{i+1}) = \lceil \frac{k}{2} \rceil - 1$ for two degree-2 vertices v_i and v_{i+1} , then $G + v_iv_{i+1}$ is balanced and TRICONN gives an optimal solution.*

The detailed procedure is given in Algorithm 5.

The Final Balancing Case

The cases that we have discussed until now give us the following entry condition on the maximum-degree face F : $t(v_i, v_{i+1}) \leq \lceil k/2 \rceil - 2$ for all i , there are at least five degree-2 vertices on the boundary of F and for a fixed degree-2 vertex u on the boundary of face F there exists no degree-2 vertex u' satisfying $|t(u, u') - t(u', u)| \leq 1$. Under these conditions, we show that it is possible to pick five degree-2 vertices from the boundary of F such that they represent the face, i.e., every internal augmentation edge is incident on one of these vertices. Once these vertices are chosen, we carefully analyze the sizes of the facial segments (captured by the values $t(v_i, v_{i+1})$) and accordingly add one or two edges inside F . We then show that the resulting graph is balanced, after which the resulting graph can be made triconnected using OUTERPLANARCONNECT. $\text{OPT}(G) = \lceil k/2 \rceil$ edges.

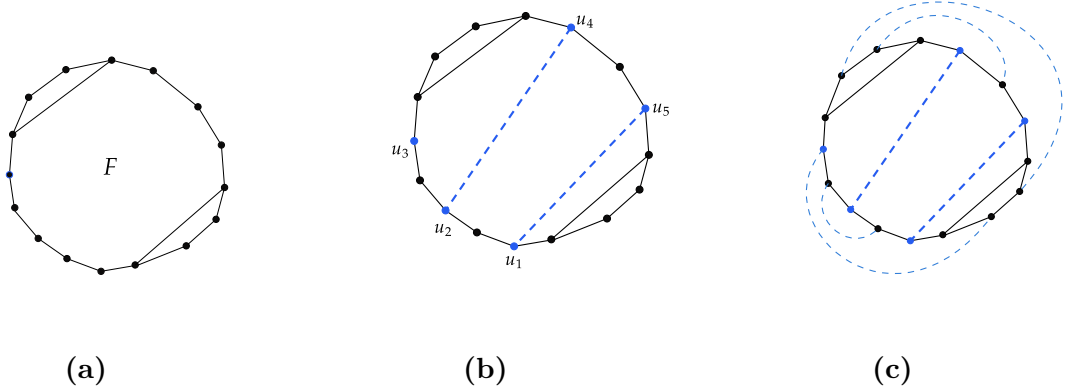


Figure 4: **(a)** The input graph with the maximum-degree face F . **(b)** Five representative degree-2 vertices selected on the boundary of F . The thick dotted blue lines show the internal augmentation edges, resulting in a balanced graph. **(c)** Augmentation of the resultant balanced graph using outer edges.

The following lemmas establish this in stages. We first show that it is always possible to select a suitable set of five vertices (Lemma 4.23). We then analyze how adding one or two edges inside the face affects the degrees of the resulting faces (Lemmas 4.25 and 4.26). Finally, we combine these results to argue that the configurations considered by FIVEVERTICES always lead to a balanced graph (Proposition 4.27). Refer to Figure 4 for an illustrative example.

Lemma 4.23. *The five degree-2 vertices $u_1, \dots, u_5$ selected by the balancing procedure satisfy*

$$t(u_i, u_{i+1}) \leq \left\lceil \frac{k}{2} \right\rceil - 2$$

for every consecutive pair.

Proof. Since there is no degree-2 vertex u' on F with $|t(u, u') - t(u', u)| \leq 1$, the balanced split of F from u must be achieved by a boundary vertex v of degree at least three. We can make the following observation about v : since k is even and $t(u, v) + t(v, u) < d(F) \leq \left\lceil \frac{k}{2} \right\rceil$ such that $|t(u, v) - t(v, u)| \leq 1$, we can conclude that $t(u, v) \leq \left\lceil \frac{k}{2} \right\rceil - 1$ and $t(v, u) \leq \left\lceil \frac{k}{2} \right\rceil - 1$. Let w be the last degree-2 vertex before v in clockwise order, and let x be the first degree-2 vertex after v we can say that $t(u, w) \leq \left\lceil \frac{k}{2} \right\rceil - 2$ (since $t(u, w) < t(u, v)$) and $t(x, u) \leq \left\lceil \frac{k}{2} \right\rceil - 2$ (since $t(x, u) < t(v, u)$). We also know that $t(w, x) \leq \left\lceil \frac{k}{2} \right\rceil - 2$ since w and x are consecutive degree-2 vertices. Since F contains at least five degree-2 vertices, pick degree-2 vertices y and z arbitrarily. These 5 vertices are identified as $\{u_i : i \in [5]\}$ in cyclic order. For every consecutive pair among these five vertices, the corresponding boundary interval is contained in one of the intervals already bounded above. From the said arguments, we obtain $t(u_i, u_{i+1}) \leq \left\lceil \frac{k}{2} \right\rceil - 2$. $\square$

Recall that all internal augmentation edges are added inside the maximum-degree face F . To ensure this, we first establish that every other face has sufficiently small degree to satisfy the entry condition for the remaining case analysis. The following observation shows that if some interior face violates this condition, then adding the edge $u_2 u_4$ inside F immediately balances the graph.

Observation 4.24. *If some interior face H satisfies $d(H) = \left\lceil \frac{k}{2} \right\rceil$, then adding the edge $u_2 u_4$ inside F produces a balanced graph.*

Proof. From Lemma 4.11, we gather that $d(F') \leq \left\lceil \frac{k}{2} \right\rceil$. Say the equality obtained for some face H , i.e., $d(H) = \left\lceil \frac{k}{2} \right\rceil$. After the edge u_2u_4 is added to the face F , the new faces F_1 and F_2 have $d(F_i) \leq \left\lceil \frac{k}{2} \right\rceil$. Since $d(F') \leq \left\lceil \frac{k}{2} \right\rceil$ for all other faces F' , the resultant graph is balanced. $\square$

Let us recall that for all faces F , $d(F) \leq k$. Indeed, in any tree, the number of leaves is at least the degree of the maximum degree node. Specifically in $T(G)$, $k \geq d(v_F)$ for all $v_F \in T(G)$. Let us prove specific conditions when adding one or two edges to F gives balance.

Lemma 4.25. *[Balancing after one internal edge] Let a face F of degree $a + b + 2 \leq k$ be split by adding one internal edge into faces F_1 and F_2 with $d(F_1) = a + 1$ and $d(F_2) = b + 1$, where $a \geq b$. If $a \leq b + 1$, then*

$$d(F_1) - 1 \leq \left\lceil \frac{k-2}{2} \right\rceil \quad \text{and} \quad d(F_2) - 1 \leq \left\lceil \frac{k-2}{2} \right\rceil.$$

Proof. Consider the scenario where a singular edge e is added to the degree-2 vertices to get two new faces F_1 and F_2 with degrees $d(F_1) = a + 1$ and $d(F_2) = b + 1$. So, $d(F) = a + b + 2 \leq k$. WLOG, let $a \geq b$. Then,

$$a \leq b + 1 \iff a \leq \left\lceil \frac{a+b}{2} \right\rceil \leq \left\lceil \frac{k-2}{2} \right\rceil \quad (1)$$

Hence, $d(F_1) - 1 \leq \left\lceil \frac{k-2}{2} \right\rceil$ and $d(F_2) - 1 \leq \left\lceil \frac{k-2}{2} \right\rceil$. $\square$

Lemma 4.26. *[Balancing after two internal edges] Suppose two internal edges added inside a face F create faces F_1, F_2, F_3 with degrees $a + 1, b + 2$, and $c + 1$, respectively, where $a + b + c + 4 \leq k$. If*

$$a \leq \left\lceil \frac{a+b+c}{2} \right\rceil, \quad b + 1 \leq \left\lceil \frac{a+b+c}{2} \right\rceil, \quad c \leq \left\lceil \frac{a+b+c}{2} \right\rceil,$$

then all new faces satisfy $d(F_i) - 1 \leq \left\lceil \frac{k-4}{2} \right\rceil$.

Proof. Consider the scenario where the edges e_1 and e_2 are added to obtain faces F_1, F_2, F_3 with degrees $a + 1, b + 2, c + 1$. For the resultant graph to be balanced, we have to check the degrees of 3 faces.

$$d(F_1) - 1 = a \leq \left\lceil \frac{a+b+c}{2} \right\rceil \quad (2)$$

$$d(F_2) - 1 = b + 1 \leq \left\lceil \frac{a+b+c}{2} \right\rceil \quad (3)$$

$$d(F_3) - 1 = c \leq \left\lceil \frac{a+b+c}{2} \right\rceil \quad (4)$$

We know that $k \geq a + b + c + 4 = d(F)$. These conditions are sufficient to obtain $d(F_i) - 1 \leq \left\lceil \frac{k-4}{2} \right\rceil$, that is, these are sufficient conditions to obtain a balanced graph. $\square$

We have built machinery to understand how edge additions affect the degree of the new faces and the balance of a graph. Let $a_i := t(u_i, u_{i+1})$ and assume $a_1 = \max_i a_i$. What follows is a lemma that affirms that the graph is balanced after the various edge additions on the different possible configurations in FIVEVERTICES.

Proposition 4.27. *[FIVEVERTICES cases yield balance] Let $u_1, \dots, u_5$ and $a_1, \dots, a_5$ be as in Lemma 4.23, and assume $a_i \leq \left\lceil \frac{k}{2} \right\rceil - 2$ for all i . In each of Case of FIVEVERTICES, the edges inserted satisfy the hypotheses of Lemma 4.25 or Lemma 4.26. Therefore, each case produces a balanced graph.*

Proof. We exhaustively consider the various cases as follows:

Case 1: $a_4 + a_3 \leq a_1 + a_2 + a_5$ and $a_2 + a_5 \leq a_1 + a_3 + a_4$.

Edges u_1u_5 and u_2u_4 are inserted, creating three faces. Using $\sum_{i=1}^5 a_i + 5 = d(F) \leq k$ and the case inequalities, we obtain

$$d(F_i) - 1 \leq \left\lceil \frac{k-4}{2} \right\rceil \quad \text{for } i = 1, 2, 3.$$

Thus, the hypotheses of Lemma 4.26 hold.

Case 2: $a_4 + a_3 = a_1 + a_2 + a_5 + 1$.

The edge u_2u_4 is inserted, splitting F into two faces. The case condition implies $a_4 + a_3 + 1 \leq \left\lceil \frac{k-2}{2} \right\rceil$, so Lemma 4.25 applies.

Case 3: $a_2 + a_5 = a_1 + a_3 + a_4 + 1$.

Edges u_5u_4 and u_1u_3 are inserted, creating three faces F_1, F_2, F_3 . From the case assumption and $a_1 = \max_i a_i$, we obtain

$$d(F_1) - 1 \leq a_2 + a_3 + 1, \quad d(F_2) - 1 \leq a_1 + a_4 + 1, \quad d(F_3) - 1 \leq a_5.$$

Using $a_2 + a_5 = a_1 + a_3 + a_4 + 1$ and $\sum_{i=1}^5 a_i + 1 = d(F) \leq k$, we derive

$$\max_i (d(F_i) - 1) \leq a_2 + a_5 \leq \left\lceil \frac{k-4}{2} \right\rceil.$$

Thus the hypotheses of Lemma 4.26 hold.

Case 4: $a_4 + a_3 \geq a_1 + a_2 + a_5 + 2$.

Edges u_1u_5 and u_2u_3 are inserted. The resulting faces satisfy

$$d(F_1) - 1 \leq a_1, \quad d(F_2) - 1 \leq a_2 + a_4 + a_5 + 1, \quad d(F_3) - 1 \leq a_3.$$

Since $a_1 = \max_i a_i$ and $\sum_{i=1}^5 a_i + 1 = d(F) \leq k$, the case inequality implies

$$a_2 + a_4 + a_5 + 1 \leq \left\lceil \frac{k-4}{2} \right\rceil.$$

Hence all faces satisfy $d(F_i) - 1 \leq \left\lceil \frac{k-4}{2} \right\rceil$, and Lemma 4.26 applies.

Case 5: $a_2 + a_5 \geq a_1 + a_3 + a_4 + 2$.

Edges u_1u_2 and u_5u_4 are inserted. The resulting faces satisfy

$$d(F_1) - 1 \leq a_2, \quad d(F_2) - 1 \leq a_1 + a_3 + a_4 + 1, \quad d(F_3) - 1 \leq a_5.$$

Using the case inequality and $\sum_{i=1}^5 a_i + 1 = d(F) \leq k$, we obtain

$$a_1 + a_3 + a_4 + 1 \leq \left\lceil \frac{k-4}{2} \right\rceil.$$

Thus all faces satisfy $d(F_i) - 1 \leq \left\lceil \frac{k-4}{2} \right\rceil$, and Lemma 4.26 applies.

In all five cases the hypotheses of Lemma 4.25 or Lemma 4.26 are satisfied, and therefore the resulting graph is balanced. $\square$

Observation 4.28. *All edges inserted by FIVEVERTICES can be drawn inside F without crossings; hence the embedding and planarity is preserved.*

Remark 4.29. There exists an optimal solution that adds at most 2 internal edges. However, there are instances where at least 2 internal edges are required to be added.

We can now finally prove the main theorem.

Theorem 4.14. *There exists an algorithm that solves OTA-Fix by running in time $\mathcal{O}(|V|(1 + \alpha(|V|)))$ time and linear space, augmenting $\text{OPT}(G) \in \{\lceil k/2 \rceil, \lceil k/2 \rceil + 1\}$ edges, where α denotes the inverse of the Ackermann function.*

Proof. Let G be the input graph with k degree-2 vertices. By Lemma 4.19, we may assume k is even. Any augmentation requires at least $\lceil \frac{k}{2} \rceil$ edges, and in some configurations at least $\lceil \frac{k}{2} \rceil + 1$ edges (Proposition 4.20).

Let F be a maximum-degree face and $v_1, \dots, v_r$ (with $v_i \bmod r$) be the degree-2 vertices in F in consecutive cyclic order.

Case 1: $T(G)$ is balanced. Then OUTERPLANARCONNECT produces a solution of size $\lceil \frac{k}{2} \rceil$, which is optimal (Lemma 4.13).

Case 2: $T(G)$ is unbalanced.

Case 2.1: F contains at most four degree-2 vertices. By Lemma 4.21, TRICONNEXACT returns an optimal solution.

Case 2.2: F contains at least five degree-2 vertices.

Subcase 2.2.1: There exist consecutive degree-2 vertices v_i, v_{i+1} with $t(v_i, v_{i+1}) \geq \lceil \frac{k}{2} \rceil$. Then at least $\lceil \frac{k}{2} \rceil + 1$ edges are required (Proposition 4.20), and TRICONN computes an optimal solution.

Subcase 2.2.2: There exist consecutive degree-2 vertices v_i, v_{i+1} with $t(v_i, v_{i+1}) = \lceil \frac{k}{2} \rceil - 1$. Then Observation 4.22 completes the augmentation using $\lceil \frac{k}{2} \rceil$ edges.

Subcase 2.2.3: For all consecutive degree-2 vertices v_i and v_{i+1} , we have $t(v_i, v_{i+1}) \leq \lceil \frac{k}{2} \rceil - 2$. By Lemma 4.23, the subroutine FIVEVERTICES selects five suitable degree-2 vertices in F . Observation 4.24 ensures that for the remaining case analysis, all the internal faces apart from F have sufficiently small degree. Coupled with the guaranties from Lemmas 4.25 and 4.26, Proposition 4.27 shows that the edges inserted by FIVEVERTICES produce a balanced graph. The remaining augmentation is then optimally completed by OUTERPLANARCONNECT.

In all cases, the number of edges inserted equals the minimum possible. Embedding preservation follows from Observation 4.28.

Running Time and Space. All structural preprocessing steps, including the construction of $T(G)$ and identification of the maximum-degree face, can be performed in linear time and space. The algorithm then performs a constant number of structural tests on face F , each requiring at most linear time. In the small-face case, the brute-force step examines only a constant number of edge insertions. The checks for large arcs and balanced split pairs are linear-time operations.

In the remaining case, FIVEVERTICES selects five vertices and inserts at most two internal edges, which takes linear time. Once the graph becomes balanced, the remaining augmentation is performed by OUTERPLANARCONNECT, which runs in $\mathcal{O}(|V(G)|(1 + \alpha(|V(G)|)))$ time (Lemma 4.13). The procedure TRICONN is invoked only in configurations where at least $\lceil \frac{k}{2} \rceil + 1$ edges are required, and by Lemma 4.9 also runs in $\mathcal{O}(|V(G)|(1 + \alpha(|V(G)|)))$ time. No step allocates more than linear auxiliary data structures, and all subroutines operate in linear space. Hence, the total space usage is $\mathcal{O}(|V(G)|)$. $\square$

5 Conclusion

We studied both edge- and vertex-connectivity augmentation under fixed planar embeddings. For 2-edge-connectivity augmentation of plane graphs (PECA-FIX), we re-introduced and used the *bt*-tree structure and a labelling scheme that identifies safe edges, yielding a near-linear-time optimal algorithm. For 3-vertex-connectivity augmentation of biconnected outerplane graphs (OTA-FIX), we used the dual tree of inner faces and related biconnecting this tree to triconnecting the original graph. Furthermore, we developed a face-splitting technique to obtain a (+1)-approximate solution and got rid of the (+1) factor with extensive case analysis and new combinatorial insights and thus obtained a near-linear-time solution.

Several questions remain open. Is it possible to solve PECA-FIX with a better running time? Can a 2-edge-connected outerplane graph be augmented to 3-edge-connectivity in near-linear or better time? Our techniques do not extend directly, but may offer useful insights. It would be interesting to extend the problem to a weighted setting where the solution edges incur a cost.

References

- [1] Hugo A. Akitaya, Justin Dallant, Erik D. Demaine, Michael Kaufmann, Linda Kleist, Frederick Stock, Csaba D. Tóth, and Torsten Ueckerdt. The Price of Connectivity Augmentation on Planar Graphs. In Vida Dujmović and Fabrizio Montecchiani, editors, *33rd International Symposium on Graph Drawing and Network Visualization (GD 2025)*, volume 357 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 23:1–23:24, Dagstuhl, Germany, 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.GD.2025.23>, doi:10.4230/LIPIcs.GD.2025.23.
- [2] M. Abellanas, A. García, F. Hurtado, J. Tejel, and J. Urrutia. Augmenting the connectivity of geometric graphs. *Computational Geometry*, 40(3):220–230, 2008. URL: <https://www.sciencedirect.com/science/article/pii/S0925772107000934>, doi:10.1016/j.comgeo.2007.09.001.
- [3] Marwan Al-Jubeih, Mashhood Ishaque, Kristóf Rédei, Diane L. Souvaine, and Csaba D. Tóth. Tri-edge-connectivity augmentation for planar straight line graphs. In Yingfei Dong, Ding-Zhu Du, and Oscar Ibarra, editors, *Algorithms and Computation*, pages 902–912, Berlin, Heidelberg, 2009. Springer Berlin Heidelberg. doi:10.1007/978-3-642-10631-6_91.
- [4] Reinhard Diestel. *Graph Theory*. Springer Publishing Company, Incorporated, 5th edition, 2017.
- [5] Kapali P. Eswaran and R. Endre Tarjan. Augmentation problems. *SIAM Journal on Computing*, 5(4):653–665, 1976. arXiv:<https://doi.org/10.1137/0205044>, doi:10.1137/0205044.
- [6] Sergej Fialko and Petra Mutzel. A new approximation algorithm for the planar augmentation problem. In *Proceedings of the Ninth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA '98*, page 260–269, USA, 1998. Society for Industrial and Applied Mathematics. URL: <http://dl.acm.org/citation.cfm?id=314613.314714>.
- [7] A. García, F. Hurtado, M. Noy, and J. Tejel. Augmenting the Connectivity of Outerplanar Graphs. *Algorithmica*, 56(2):160–179, February 2010. doi:10.1007/s00453-008-9167-1.

- [8] Carsten Gutwenger, Petra Mutzel, and Bernd Zey. On the hardness and approximability of planar biconnectivity augmentation. In Hung Q. Ngo, editor, *Computing and Combinatorics*, pages 249–257, Berlin, Heidelberg, 2009. Springer Berlin Heidelberg. doi:10.1007/978-3-642-02882-3_25.
- [9] Carsten Gutwenger, Petra Mutzel, and Bernd Zey. Planar biconnectivity augmentation with fixed embedding. In Jiří Fiala, Jan Kratochvíl, and Mirka Miller, editors, *Combinatorial Algorithms*, pages 289–300, Berlin, Heidelberg, 2009. Springer Berlin Heidelberg. doi:10.1007/978-3-642-10217-2_29.
- [10] Tanja Hartmann, Jonathan Rollin, and Ignaz Rutter. Cubic augmentation of planar graphs. In Kun-Mao Chao, Hsu-Chun Yen, and In-Bok Yeom, editors, *Algorithms and Computation - 23rd International Symposium, ISAAC 2012, Taipei, Taiwan, December 19-21, 2012. Proceedings*, volume 7676 of *Lecture Notes in Computer Science*, pages 266–276. Springer, 2012. doi:10.1007/978-3-642-35261-4_43.
- [11] Tanja Hartmann, Jonathan Rollin, and Ignaz Rutter. Regular augmentation of planar graphs. *Algorithmica*, 73(2):306–370, 2015. doi:10.1007/s00453-014-9922-4.
- [12] T.-S. Hsu and V. Ramachandran. A linear time algorithm for triconnectivity augmentation. In [1991] *Proceedings 32nd Annual Symposium of Foundations of Computer Science*, pages 548–559, 1991. doi:10.1109/SFCS.1991.185418.
- [13] Tsan-sheng Hsu. On four-connecting a triconnected graph. *J. Algorithms*, 35(2):202–234, May 2000. doi:10.1006/jagm.2000.1077.
- [14] Bill Jackson and Tibor Jordán. Independence free graphs and vertex connectivity augmentation. *Journal of Combinatorial Theory, Series B*, 94(1):31–77, 2005. URL: <https://www.sciencedirect.com/science/article/pii/S0095895604001224>, doi:10.1016/j.jctb.2004.01.004.
- [15] Goos Kant. Augmenting outerplanar graphs. *J. Algorithms*, 21(1):1–25, July 1996. doi:10.1006/jagm.1996.0034.
- [16] Goos Kant and Hans L. Bodlaender. Planar graph augmentation problems. In Frank Dehne, Jörg-Rüdiger Sack, and Nicola Santoro, editors, *Algorithms and Data Structures*, pages 286–298, Berlin, Heidelberg, 1991. Springer Berlin Heidelberg. doi:10.1007/BFb0028270.
- [17] Hiroshi Nagamochi and Peter Eades. Edge-splitting and edge-connectivity augmentation in planar graphs. In Robert E. Bixby, E. Andrew Boyd, and Roger Z. Ríos-Mercado, editors, *Integer Programming and Combinatorial Optimization*, pages 96–111, Berlin, Heidelberg, 1998. Springer Berlin Heidelberg. doi:10.1007/3-540-69346-7_8.
- [18] Ignaz Rutter and Alexander Wolff. Augmenting the connectivity of planar and geometric graphs. *Electronic Notes in Discrete Mathematics*, 31:53–56, 2008. The International Conference on Topological and Geometric Graph Theory. URL: <https://www.sciencedirect.com/science/article/pii/S1571065308000656>, doi:10.1016/j.endm.2008.06.009.
- [19] Csaba D. Tóth. Connectivity augmentation in planar straight line graphs. *European Journal of Combinatorics*, 33(3):408–425, 2012. Topological and Geometric Graph Theory. URL: <https://www.sciencedirect.com/science/article/pii/S0195669811001508>, doi:10.1016/j.ejc.2011.09.002.

- [20] László A. Végh. Augmenting undirected node-connectivity by one. *SIAM J. Discret. Math.*, 25(2):695–718, 2011. doi:10.1137/100787507.
- [21] Toshimasa Watanabe and Akira Nakamura. Edge-connectivity augmentation problems. *Journal of Computer and System Sciences*, 35(1):96–144, 1987. URL: <https://www.sciencedirect.com/science/article/pii/0022000087900389>, doi:10.1016/0022-0000(87)90038-9.
- [22] Toshimasa Watanabe and Akira Nakamura. A minimum 3-connectivity augmentation of a graph. *Journal of Computer and System Sciences*, 46(1):91–128, 1993. URL: <https://www.sciencedirect.com/science/article/pii/0022000093900507>, doi:10.1016/0022-0000(93)90050-7.
- [23] Jeffery Westbrook and Robert E. Tarjan. Maintaining bridge-connected and biconnected components on-line. *Algorithmica*, 7(1):433–464, June 1992. doi:10.1007/BF01758773.